# Verification guide

Rank-six categorification through multiplicity eight

29 September 2026

## 1 Scope and authoritative records

This guide accompanies *Rank-six fusion rings: categorification criteria and models through multiplicity eight*. The counting input has 16612 rings, with exact-multiplicity counts $39, 154, 384, 872, 1582, 2768, 3814, 6999$. It comes from the public *Fusion Ring Census*; no independent census is performed here.

The classification ledger contains 16536 general exclusions, 53 unitary-positive rings, one spherical positive ring with no pseudo-unitary categorification, and 22 residuals. The 54 positive rings have 61 constructor records, including alternative models. General categorifiability is completely classified through multiplicity three: precisely 46 of the 577 rings in that range are categorifiable, all have spherical models, and 45 have unitary models. The other 531 rings are generally excluded. The exception to pseudo-unitary existence is Schopieray's $R^{(6)}_{2,124}$.

The authoritative files are `data/census.json`, `data/decisions.json` and `data/classification_counts.json`. The CSV ledger and manuscript tables are derived exports. The model catalogue records sources, categorical structures and explicit basis permutations; the residual export contains complete tensors and all five existence fields. A structure-specific exclusion does not change an unknown general existence field into a general exclusion.

## 2 Frozen census input

The source is *Fusion Ring Census*, revision

`b86eb6dc6eeeafe5fc5f6906d7b074e0322e9726`.

The rank-six files are `results/census.json` and `results/rank6/tables.txt.gz`. The compressed tensor SHA-256 is

`0138c6228dbb06b0861ad38b8dfe67fcd1fc829add80d441c34c09a1afb78675`.

The uncompressed tensor SHA-256 is

`7898a833dfffd0f645436c1d326740db29ea6a29980de307c67227f0b67122b0`.

The bundled tensor file is `data/rank6_tables.txt.gz`; `provenance/repository_source.json` records its source and hashes. Import, classification and checking run offline from these files. The pinned source supplies the completeness claim. Import checks fix the input and its correspondence with the classification registry; they do not re-enumerate rings or turn a hash into a completeness proof. The established representatives and identifiers through multiplicity seven are preserved. Exact-multiplicity-eight representatives are sorted in the same row-major convention. Every obstruction and model comparison operates on full tensors, never on an identifier alone.

# 3 Automated classification and exact checking

The manuscript orders exclusion families to expose inexpensive structural and arithmetic tests before larger finite searches. Its sequential table counts each removal once and reports what remains at every exact multiplicity. This order balances observed usefulness with transparency; it is not a proof of a universally optimal runtime. The first unresolved multiplicity identifies the next stratum requiring work. A zero residual count at some larger tested multiplicity says nothing by itself about untested multiplicities.

Use Python with the versions in `requirements.txt`, a C++17 compiler for the finite-search helpers, and `pdflatex` for the PDFs. Native helpers are compiled from their supplied source as needed. The verification commands are:

```
python -m pip install -r requirements.txt
python check_manifest.py
python verify.py --jobs 2 --out verification/current/recheck.jsonl
python -O verify.py --jobs 2 --out verification/current/recheck_O.jsonl
python software/current_regressions.py
```

For a fresh discovery campaign, first copy the archive to a separate working directory. The following commands overwrite its campaign files; they are not needed to verify the frozen release:

```
python software/import_repository.py
python software/classify.py --jobs 4
```

Reports are stored in `verification/classification/`. The files `stages.json` and `summary.json` record actual stage counts and outcomes. The adjacent `attempts.jsonl` distinguishes failed or unfinished attempts from accepted certificates. Discovery can be more expensive than checking existing certificates. These commands apply categorification criteria; they do not enumerate new fusion rings.

Run the manifest check before writing reports: a new report changes file bytes. The normal and optimized Python modes check that the mathematical checks do not depend on removable assertions. A failed subprocess must have a nonzero return code and cannot be reported as a successful check. The saved reports state their input, scope and actual outcomes; an older report does not establish a result for a larger census.

A negative certificate is accepted only after its checker reconstructs the required number fields, finite alternatives, separating witnesses and exact inequalities from the original tensor. Numerical linear and mixed-integer optimization may assist discovery, but their entry points are disabled during negative-certificate checking. A timeout, ineligibility, numerical infeasibility flag or missing certificate leaves the case unresolved. A successful necessary relaxation also leaves it unresolved unless a categorical construction is available.

The option `--proofs-only` retains source-integrity checks but omits positive-model reconstruction; it is not a complete package check. Regression controls cover native/reference agreement and deliberately corrupted certificates. Saved verification is computer-assisted exact arithmetic, not proof-assistant formalization: the categorical input theorems and their hypotheses remain mathematical inputs to the argument.

# 4 Necessary criteria and their certificates

## 4.1 Dimension order and character constraints

`software/dimension_order.py` computes the $\mathbb{Z}$-span of simple dimensions in a rational power basis using Hermite normal form. The necessary condition is membership of the required

codegree quotient in this actual order, not merely in the maximal order. The checker verifies the codegree root, lattice, separating functional and nonintegral value of the functional on the quotient. General wrappers exhaust all required oriented signed tables and dimension-character alternatives. Failure for only one possible character is not general nonexistence.

Character-aware induction uses joint eigenvalue labels, reciprocity and Galois restrictions in the same central-induction system. A discarded column must violate a proved necessary condition. Existing exact separating vectors remain valid after such columns are removed, because the inequalities on the retained columns are unchanged. The certificate allocation records the accepted wrapper as well as its inner tests.

### 4.2 Signed spherical traces

`software/signed_spherical_traces.py` reconstructs signed characters and low-column searches. It keeps integer restriction multiplicities, Gram positivity, signed indicator equalities, joint Galois closure and modular equations. It does not replace signed categorical dimensions with absolute values or import pseudo-unitary mass bounds without a valid reduction. The stored certificate for $R^{(6)}_{4,446}$ exhausts 290 low matrices, two surviving low frames, two character labellings per frame and all 32 second-indicator choices per labelling; its 128 separating vectors are checked exactly.

### 4.3 Module, subring and Morita reductions

Subring descent uses the published or cited categorification statements with their stated ranks, multiplicities and categorical assumptions. A larger counting census does not enlarge the range of a lower-rank categorification theorem. Module-Gram and Morita constraints likewise require the module or subcategory hypotheses proved in the manuscript. Their applicability is checked before a certificate is accepted.

### 4.4 The two continuation files and release workflow

The original sixteen-stage campaign is preserved in `verification/classification/`, including its unchanged accepted-certificate file. The first continuation used forty-seven frozen inputs and accepted seven general exclusions in `certificates/continuation_20260929.jsonl`. Its bounded discovery outcomes and replay evidence are in `verification/continuation/`. These are historical records with their own ledger and input hashes.

The second continuation begins with forty tensors fixed in `verification/round2/campaign_manifest.json`: the last multiplicity-three residual and all thirty-nine remaining tensors of multiplicities four through eight. This file preserves full tensors, identifiers and input hashes. The criterion scans and higher stabilizer campaign use this domain rather than a residual list that shrinks after new exclusions. A recorded ineligibility, timeout or surviving necessary system is a research outcome, not a negative certificate.

Only general certificates accepted by the checker enter `certificates/continuation_round2.jsonl`. The release is the disjoint union of the original campaign and both continuation files. Alternative proofs of the same tensor are verified but not counted twice. The following commands perform the second freeze and an independent replay of the accepted continuation records:

```
python -O software/round2_release.py
python -O software/continuation_controls.py --jobs 2 \
  --out verification/round2/replay_optimized.json
```

The first command reads the canonical general proof sources, checks their tensors against the frozen input manifest, disables numerical optimization, verifies alternatives and all accepted

continuation certificates, writes the second JSONL file and rebuilds the ledger in one process. Its receipt is `verification/round2/release_replay.json`. The second command checks both continuation files against the current ledger and replays each certificate independently, again with both linear and mixed-integer optimizers disabled. The supplied output argument keeps its report separate from the first continuation's historical reports. Omitting `-O` gives the corresponding normal-Python check; report filenames and hashes identify which invocation actually ran.

Neither command reruns all unchanged original negative certificates. The full-ledger `verify.py` commands above do that and also check the source and positive models, unless explicitly restricted by `--proofs-only`. Retained incremental evidence is not described as a newly executed full replay.

The first continuation's three additional general obstructions are implemented by `new_rank3_subring.py`, `new_module_spectral.py`, and `new_morita_order6.py` in `software/`. Their checkers reconstruct the closed subring or the entire finite matrix/ring domain from the input tensor. The order-six reduction checks all 60 cyclic and four nonabelian possibilities, including inherited subring certificates and the explicit faithful gradings. The revised general character-induction exclusions are accepted only with complete signed/character branch coverage and exact separating witnesses.

The full Morita-action Gram criterion is in `software/full_morita_gram.py`. Its first-continuation residual scan alone yielded no additional whole-ring exclusion. The second continuation couples complete Morita actions to the local rotation constraint; that stronger argument has its own general certificate. A feasible rational Gram system is only a necessary condition; the exploratory finite-completion scripts are not used to allocate a status from a numerical infeasibility flag.

### 4.5 Historical center compatibility and the final obstruction

The command

```
python research/center_continuation/exact_cyclic8_center.py
```

checks the frozen rank-22 center tensor and modular datum in the exact cyclotomic quotient $\mathbb{Z}[x]/(\Phi_{136}(x))$. It verifies all 10 648 spectral fusion identities, associativity, all 2 904 restriction-ring identities and all second indicators. The same command with Python's `-O` flag checks without assertions. This is a compatibility certificate for $R^{(6)}_{3,215}$, not a categorical model: the required connected commutative separable algebra and tensor functor are not constructed by that compatibility test. A separate source-specific Morita argument now rules out lifting this displayed map from the actual known center. The audit in `research/round2_audit/actual_cyclic8_morita.md` strengthens that source-specific argument to the entire actual cyclic-eight Morita class by forcing the canonical unit-induction support and module traces. Neither argument classifies all braided categories having the same modular data. The new local involution-rotation obstruction independently gives a general exclusion of the target ring; it does not identify a braided category from these modular data.

The additional script `research/new_frontier/frontier_spherical_reduction.py` checks unrestricted signed sphericalization: 1480 quotients in 63 sign-gauge orbits, with every nongauge orbit excluded by exact zero or norm tests. The surviving gauges give a spherical structure, and their only nonzero dimension-character orbit is the Frobenius–Perron orbit. Thus general existence for this one ring is equivalent to pseudo-unitary existence. This reduction is one required component of the final general exclusion, together with the full Morita-action calculation and the stabilizer-rotation certificate described below.

### 4.6 Biquadratic dimension-field traces

`software/new_biquadratic_trace.py` implements the general trace obstruction whose accepted records are in `certificates/biquadratic_exclusions.json`. Central induction in the sphericalization forces its dimension field to be the biquadratic field generated by the original formal codegrees. Character orthogonality, the integral basis and the Casimir trace bound give a finite domain of totally positive possible dimensions. The $d$-number norm condition and the two remaining signed codegrees reduce that domain further. Finally, the signed and original Casimir characteristic polynomials must agree modulo two.

The checker reconstructs these finite domains from the full tensor. The frozen-input scan covers all forty inputs, including all thirty-nine higher-multiplicity tensors. A separate scan checks all 16612 census tensors and yields fifty-one independent exclusions, with counts $1, 6, 11, 15, 18$ at multiplicities four through eight. These overlapping criterion totals are not new-ledger allocations. All 54 established positive models pass the necessary test. The independent scope audit is in `research/round2_literature/biquadratic_trace_audit.md`; the optimized exact replay, imported-example regression and corrupted-certificate controls are recorded in `research/round2_external/biquadratic_trace_audit.json`. The proof uses the sphericalization of an arbitrary category, so it does not require a pivotal assumption on the original categorification.

### 4.7 Ramification, trace ideals and all-orders Gauss bounds

`software/new_ramified_signed.py` reconstructs the mod-two algebra, its primitive factors and the formal-codegree integrality constraints. A ramified quadratic dimension field can be isolated by these local factors. The resulting Casimir bound either contradicts the tensor directly or bounds a complete signed search. A successful signed reduction to the FP orbit must still be combined with a verified pseudo-unitary obstruction before receiving a general exclusion status. The uniform frozen-input report is `research/round2_higher/ramification_application.json`; it includes every higher input and explicit failures of applicability.

`software/new_trace_ideal.py` does not require isolation of the dimension field. For each original rational or quadratic codegree $f$, relative trace forces the global-dimension trace into $\mathrm{Tr}_{\mathbb{Q}(f)/\mathbb{Q}}(f\mathcal{O}_{\mathbb{Q}(f)})$. Their least common multiple cannot exceed the original Casimir trace. The same bound can force every signed coefficient to retain its full absolute value, leaving an exhaustive sign search. Its full-census scan checks all 16612 tensors; 11241 satisfy the implemented self-dual applicability tests, and 1827 are independently excluded. The corresponding exclusion counts at multiplicities four through eight are $115, 238, 317, 443, 634$. These numbers include previously excluded rings. The full reports and optimized controls are `research/round2_external/trace_ideal_full_census_scan.json`, `trace_ideal_full_census_summary.json`, and `trace_ideal_audit.json` in that directory.

`software/new_gauss_family.py` recognizes a rank-six family by its full tensor. It exhausts the low and high central restriction columns and then uses a quadratic Gauss-sum bound on roots of unity of all orders; no conductor cutoff is substituted for that bound. Its inner certificate has pseudo-unitary scope. For $R^{(6)}_{8,6520}$, the trace-ideal saturation reduction and this inner certificate form the general wrapper in `research/round2_external/R6_8_6520_general.json`. Its exact replay requires 4298 roots where the Gram and twist equations permit only 590. The family scan covers the entire census and finds one applicable member at each multiplicity four through eight; those pseudo-unitary results are not automatically general exclusions without their recorded reductions.

The imported proofs in `research/round2_external/sources/` are retained with their original tensors and audit. The R3582 input matches $R^{(6)}_{7,3582}$ exactly; its release certificate uses the generic ramification reduction and a generic induction obstruction. The B13 input

has multiplicity thirteen, supplies a regression test for the biquadratic arithmetic, and adds no tensor to the census.

The uniform arithmetic scans and controls can be reproduced by

```
python -O research/round2_external/audit_biquadratic_trace.py
python -O research/round2_external/audit_trace_ideal.py
python -O software/new_biquadratic_trace.py --all \
  --output research/round2_external/biquadratic_full_census_scan.json
python -O software/new_trace_ideal.py --all \
  --output research/round2_external/trace_ideal_full_census_scan.json
python -O research/round2_higher/verify_ramified.py --audit
python -O research/round2_higher/verify_gauss_family.py
```

These commands rewrite their stated research reports and, for the audit scripts, their canonical source records; run them in a working copy after checking the archive manifest. They do not modify the census. Re-freezing the release after source-record regeneration uses `software/round2_release.py`.

### 4.8 The complete multiplicity-three obstruction

The manuscript's stabilizer-rotation theorem combines four independently necessary steps: exhaustive signed sphericalization, full two-sided Morita actions for the algebra $1 + g$, the local involution-rotation lemma, and exhaustive central-induction separation. The module action forces the mixed left/right stabilizer commutator to be $-1$ for the two $g$-fixed simples. Adjacent pair-insertion operators therefore anticommute on their two-dimensional cubic invariant spaces. Naturality of pivotal rotation makes the rotation nonscalar; the Cauchy theorem and cyclotomic field intersection force the third indicators to be $-1$.

`software/stabilizer_rotation.py` implements both the positive-spherical inner criterion and the complete general wrapper. The certificate `certificates/stabilizer_rotation_215.json` replays through `verify_general(R,c)`. It reconstructs all 1480 signed quotients, 63 sign-gauge orbits, and 271 required nongauge character rejections; it then checks all 16 involution-module branches, 91 low induction frames, and 200 exact height-three separating rays. The reduction is implemented in `software/galois_pseudo_reduction.py`. The local proof and source audit are in `research/round2_literature/local_involution_rotation.md`.

The direct replay commands read the existing certificate without regenerating it:

```
python software/stabilizer_rotation.py R6_3_215 \
  --output certificates/stabilizer_rotation_215.json --verify
python -O software/stabilizer_rotation.py R6_3_215 \
  --output certificates/stabilizer_rotation_215.json --verify
```

The general record is also checked by the top-level verifier. Its checker reconstructs the finite branch domains and uses exact integer arithmetic for every separating vector. An isolated positive-spherical inner certificate would not supply the required general conclusion.

### 4.9 Higher stabilizer and quartic ambient-field refinements

`software/stabilizer_rotation_higher.py` extends the local Clifford constraint to larger cubic invariant spaces, including eligible non-self-dual simples. It checks every involution-module possibility. For a negative stabilizer commutator it first imposes the proved indicator interval, then, when needed, all discrete averaged values. Every accepted branch has an exact separating ray. Its inner scope is pseudo-unitary; the general wrapper verifies the independent signed reduction from `software/galois_pseudo_reduction.py`.

`research/round2_theory/higher_campaign.json` indexes the frozen forty-input campaign, including every one of the thirty-nine higher inputs. Its outcomes distinguish completed exclusions, ineligibility, surviving necessary systems and explicit budget stops. Higher trace heights are additional recorded attempts, not a claim of complete search at arbitrary height. The positive controls include the entire 54-model catalogue. The optional `software/new_signed_csp.py` and `software/new_signed_csp.cpp` are discovery helpers; unfinished unrestricted searches supply no negative records.

`software/cyclotomic_stabilizer_trace.py` handles supported quartic FP fields by verifying an embedding into a real cyclotomic field. It exhausts the proved finite relative-trace domain, including the zero trace vector accounting for all orders outside the stated box, all admissible high columns, and the required indicator choices. The inner condition combines these traces with the tensor-forced local rotation bound. The general wrapper additionally checks unrestricted signed sphericalization and all dimension-character alternatives. It does not identify a fusion category from a matching number field.

Canonical general records for these refinements occur in the `research/round2_center/`, `research/round2_theory/` and `research/round2_literature/` directories. The release script admits only supported general proof kinds from those records. Standalone necessary-system certificates, reductions without an inner obstruction, partial searches and exploratory module completions are kept out of the accepted JSONL file. No hard-coded list of ring labels replaces the tensor hypotheses or the exhaustive checking routines.

## 5 Positive models

Positive conclusions require a categorical construction and a complete tensor match. Sources include quantum-group categories, finite-group representation categories, near-group and subfactor constructions, central-boson quotients, equivariantizations and relative centers. The model catalogue preserves its bibliographic attribution and the precise categorical structure supplied by each source.

The constructors in `software/residual_models.py` identify Extended Haagerup, Izumi's $C_5 + 5$ category, the mixed half-$E_6/PSU(2)_6$ quotient and sign-swap examples. The relative Fibonacci center of $TT_3$ is reconstructed in `software/relative_fibonacci_models.py`; its restriction matrix and exact linear system are stored in `data/relative_tt3.json`. The Fibonacci inputs and categorical relative-center construction are part of the argument. A relative center is not a Müger centralizer.

The constructors in `software/through7_models.py` reconstruct the Asaeda–Haagerup $\mathcal{AH}_1$ table and the representation ring of $(C_3 \times C_3) \rtimes Q_8$. The latter constructs the order-72 group, checks its character table and obtains tensor coefficients by exact character inner products. The quantum-group, cochain, finite-group and relative-center controls all compare complete multiplication tensors.

Centrality in a fusion ring is not a categorical central lift, and an associative formal tensor is not automatically categorifiable. A numerical dimension match supplies neither the tensor match nor a category. These distinctions apply to alternative models as well as to newly resolved rings.

## 6 Residual tensors and finite witnesses

The first unresolved stratum is read from `data/unresolved_tensors.json`; there are no residuals through multiplicity three. The tensor $R^{(6)}_{3,215}$ was the last residual at that bound and is now generally excluded. Its older feasible relaxations remain archived as scoped

regression controls. Its file `systems/integral_frontier_215.json` records a feasible point for a specified central-induction relaxation. The commands

```
python software/modular_frontier.py
python -O software/modular_frontier.py
```

reconstruct the low frame, character labels, second indicators, pruning and 230 rational equations in 199 variables. The supplied point has nonnegative integral coordinates and formal central rank 22. This statement concerns that relaxation. The separate exact center completion described above now supplies compatible twists and a full center fusion ring, but still no associators for the rank-six target or an algebra structure realizing its restriction map.

### 6.1 The joint product test and literature comparisons

Before the final rotation obstruction, $R^{(6)}_{3,215}$ was the first general residual. The manuscript retains a joint fourth-order product-Gram condition for pseudo-unitary center data with specified self-dual low objects. This couples all low products in a single nonnegative integral Gram factorization and enforces their restriction equations. The checking command is

```
python research/m8_frontier/low_product_gram.py --check
```

It reconstructs the inherited induction witness, the cubic and fourth-order moments, all fifteen product restrictions and the expanded Gram factorization. The rank-22 formal point survives: the residual $15 \times 15$ Gram matrix has rank three, its three high restriction types have multiplicities $(4, 8, 4)$, and the forced per-object product coefficients are nonnegative integers. The product-Gram test alone supplies no complete center fusion tensor, individual twists or associators; these first two data are supplied by the separate continuation certificate above. Rejecting just one proposed center point would not by itself exclude the ring either.

The two literature checks are

```
python research/m8_frontier/premodular_bound.py
python research/m8_frontier/nrw_rank6_scan.py
```

The first implements the rank-six bound FPdim $\leq 168$ for braided spherical categorifications derived from the Dong–Schopieray catalogue. It checks the dimension character and the strict inequality $136 + 32\sqrt{17} > 168$ exactly for the frontier tensor. The result excludes premodular categorification only; it is not a general, pivotal, spherical, pseudo-unitary or unitary exclusion without the additional braiding hypothesis.

The second reads the frozen literal integer tensors in the Ng–Rowell–Wen rank-six ancillary data and screens all 16612 census tensors. Exact basis-permutation comparisons follow invariant filtering. The 192 modular records comprise 20 Galois orbits and eleven based-ring types, all of multiplicity at most two; there is no new model for the former multiplicity-three frontier or the multiplicity-eight stratum. The source comparison does not exclude arbitrary fusion categories. Both programs save explicit scope and source identities with their reports. No ring-specific hand arithmetic is required to reproduce any of these calculations.

The feasible witnesses for $R^{(6)}_{2,4}$ are separate controls for a ring with an independently justified positive model. They must not be transferred to a residual tensor. Exact feasible points explain the limits of the corresponding necessary systems; they do not show that all conceivable refinements would fail. A literature comparison counts as a new model only when the source actually constructs a category and its full fusion tensor agrees.

# 7 Regenerating the presentation

The manuscript has seven main source chapters, with continuation subsections included from their separate theorem files. After the second-continuation freeze and replay described above, the following commands rebuild the derived ledgers, tables and companion PDFs from the retained records:

```
python software/release.py
python software/model_catalog.py
python software/build_paper_tables.py
python build_papers.py
```

The model registry itself can be reconstructed with `python software/residual_models.py` before these commands. All generated counts must agree with the authoritative decision ledger. The eight OEIS terms count based rings rather than categorifiable rings; no accession or online submission is claimed. No census enumeration is part of this reproduction procedure.